

WEEK-4 DYNAMIC PROGRAMMING

Question 1

Question text

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3. Write any efficient algorithm to find the possible ways.

Example 1:

Input: 6

Output: 6

Explanation: There are 6 ways to 6 represent number with 1 and 3

1+1+1+1+1+1

3+3

1+1+1+3

1+1+3+1

1+3+1+1

3+1+1+1

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    if (scanf("%d", &n) != 1) return 0;
```

```
    if (n <= 0) {
```

```
        printf("0\n");
```

```
        return 0;
```

```
    }
```

```
    if (n == 1 || n == 2) {
```

```
        printf("1\n");
```

```
        return 0;
```

```

}

long long dp[n + 1];

dp[0] = 1;

dp[1] = 1;

dp[2] = 1;

for (int i = 3; i <= n; i++) {

    dp[i] = dp[i - 1] + dp[i - 3];

}

printf("%lld\n", dp[n]);

return 0;

}

```

Input	Expected	Got	
	6	6	6
	25	8641	8641
	100	24382819596721629	24382819596721629

Question 2

Correct

Mark 10.00 out of 10.00

Flag question

Question text

Playing with Chessboard:

Ram is given with an $n \times n$ chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only

travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3
1 2 4
2 3 4
8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value: $1+2+8+7+1=19$

ANSWER

```
#include <stdio.h>

#define MAX 100

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;

    if (scanf("%d", &n) != 1) return 0;

    int grid[MAX][MAX];

    int dp[MAX][MAX];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &grid[i][j]);
        }
    }

    dp[0][0] = grid[0][0];
```

```

for (int j = 1; j < n; j++) {
    dp[0][j] = dp[0][j - 1] + grid[0][j];
}
for (int i = 1; i < n; i++) {
    dp[i][0] = dp[i - 1][0] + grid[i][0];
}
for (int i = 1; i < n; i++) {
    for (int j = 1; j < n; j++) {
        dp[i][j] = grid[i][j] + max(dp[i - 1][j], dp[i][j - 1]);
    }
}
printf("%d\n", dp[n - 1][n - 1]);
return 0;
}

```

Input	Expected	Got	
3 1 2 4 2 3 4 8 7 1	19	19	
3 1 3 1 1 5 1 4 2 1	12	12	
4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	

Question 3

Question text

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1	a	g	g	t	a	b	
s2	g	x	t	x	a	y	b

The length is 4

Solving it using Dynamic Programming

ANSWER

```
#include <stdio.h>

#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char s1[1000], s2[1000];

    if (scanf("%s %s", s1, s2) != 2) return 0;

    int m = strlen(s1);
    int n = strlen(s2);

    int dp[m + 1][n + 1];

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0) {
```

```

        dp[i][j] = 0;
    } else if (s1[i - 1] == s2[j - 1]) {
        dp[i][j] = 1 + dp[i - 1][j - 1];
    } else {
        dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
    }
}
}

printf("%d\n", dp[m][n]);

return 0;
}

```

Input	Expected	Got	
	aab azb	2	2
	ABCD ABCD	4	4

Question 4

Question text

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

ANSWER

```
#include <stdio.h>
```

```
int main() {  
  
    int n;  
  
    if (scanf("%d", &n) != 1) return 0;  
  
    int arr[n];  
  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }  
  
    int dp[n];  
    int max_len = 0;  
  
    for (int i = 0; i < n; i++) {  
        dp[i] = 1;  
        for (int j = 0; j < i; j++) {  
            if (arr[i] >= arr[j] && dp[j] + 1 > dp[i]) {  
                dp[i] = dp[j] + 1;  
            }  
        }  
        if (dp[i] > max_len) {  
            max_len = dp[i];  
        }  
    }  
  
    printf("%d\n", max_len);  
  
    return 0;  
}
```

}

Input	Expected	Got	
9 -1 3 4 5 2 2 2 2 3	6	6	
7 1 2 2 4 5 7 6	6	6	